ITCS215
Introduction to Interactive Multimedia Technology

Class: 1-2025
Module III: Introduction to Computer Graphics, Generative AI, and Audio
Lecture: TUE 13:00-16:00

Unit 1: Introduction to Computer Graphics: Drawing I

Dr. Kanoksak Wattanachote (Ph.D) (金功勳 博士)
kanoksak.wat@mahidol.ac.th

Unit 1: Content

Unit 1: Introduction to Computer Graphics: Drawing I

- Introduction to Multimedia
- Computer Graphics
- History of Computer Graphics
- Color and Color Systems
- Pixel
- Drawing I

Getting Start
with

Computer Graphics



Definition

Module III: Computer Graphics

- Computer graphics is the art of drawing pictures on computer screens with the help of programming that involves
  - computations
  - creation
  - manipulation of data.

- Computer graphics is a rendering tool for the generation and manipulation of images.

Definition

Module III: Computer Graphics

- Computer graphics involves the display, manipulation, and storage of pictures and experimental data for proper visualization using a computer.
  - It provides methods for producing images and animations (a sequence of images).

Definition

Module III: Computer Graphics

- In this lecture, we explore what computer graphics is about and survey some application areas.
- Computer graphics deals with all aspects of creating images with a computer:
  - Hardware
  - Software
  - Application

Computer Graphics Application

Module III: Computer Graphics

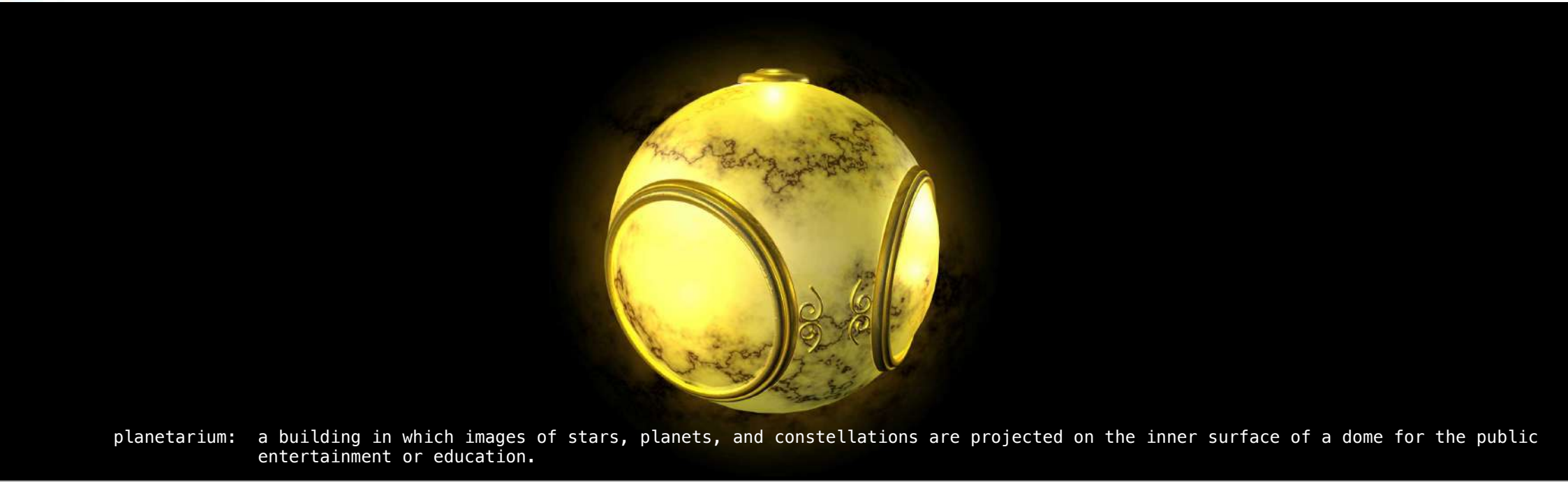
- Where did this image come from?
- What hardware/software do we need to produce it?



Computer Graphics Application

Module III: Computer Graphics

- Preliminary answer:
  - Application: The object is an artist's rendition of the sun for an animation to be shown in a domed environment (planetarium).



planetarium: a building in which images of stars, planets, and constellations are projected on the inner surface of a dome for the public entertainment or education.

Computer Graphics Application

Module III: Computer Graphics

- Preliminary answer:
  - Software: Maya for modelling and rendering, but Maya is built on top of OpenGL.



Computer Graphics Application

Module III: Computer Graphics

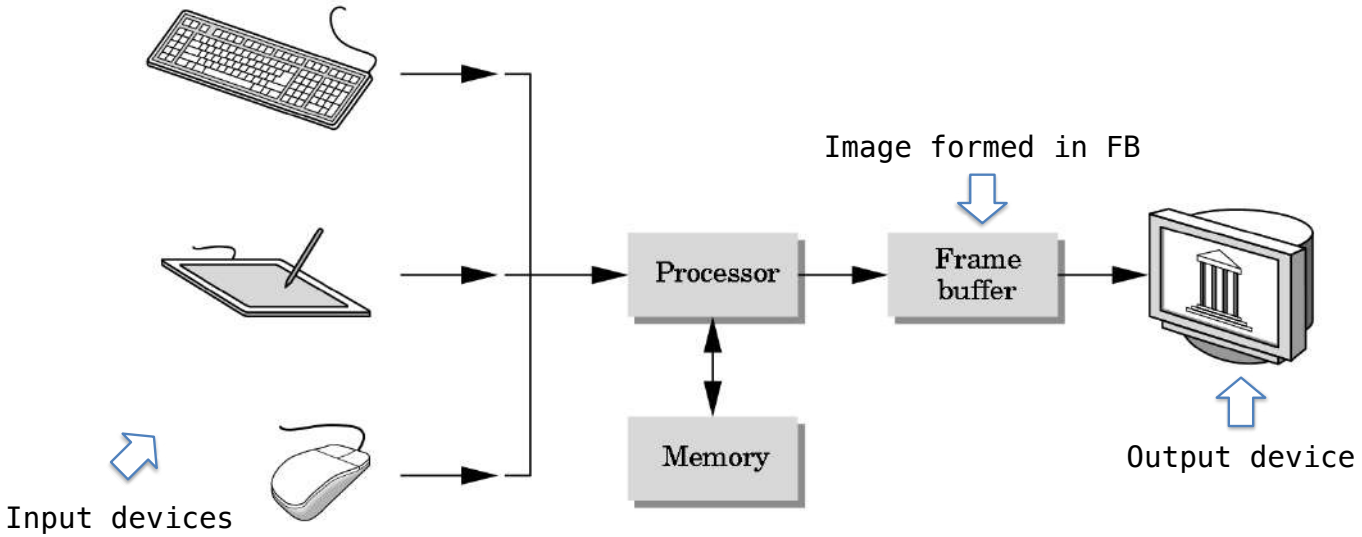
- Preliminary answer:
  - Hardware: PC with graphics card for modelling and rendering.



Basic Graphics System

Module III: Computer Graphics

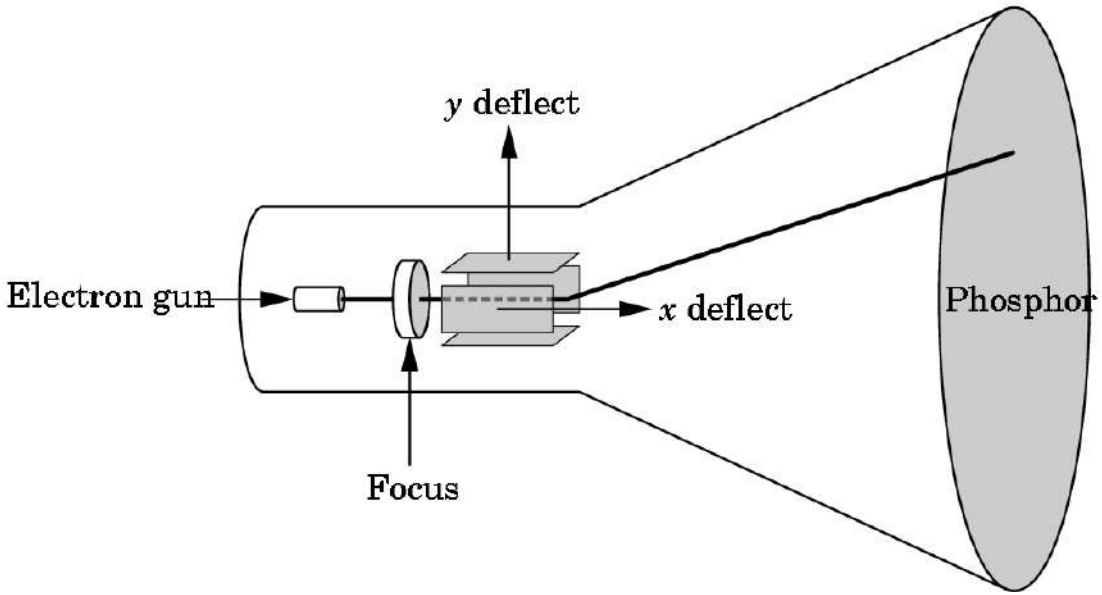
- The frame buffer can be viewed as the core element of a graphics system.
- The resolution is the number of pixels in the frame buffer.
- The depth, or precision, of the frame buffer is defined as the number of bits that are used for each pixel.



CRT

Module III: Computer Graphics

- The Cathode Ray Tube (CRT) is a vacuum tube containing an electron gun (a source of electrons) and a fluorescent screen, with internal or external means to accelerate and deflect the electron beam, used to create images in the form of light emitted from the fluorescent screen.

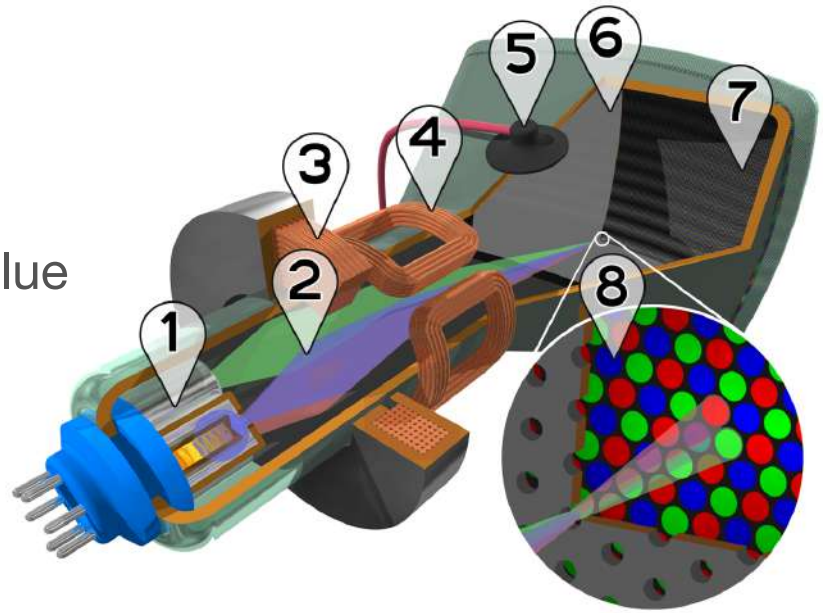


- The image may represent electrical waveforms (oscilloscope), pictures (television, computer monitor), radar targets, and others.

CRT

Module III: Computer Graphics

- Cutaway rendering of a color CRT.
  1. Three electron guns for red, green, and blue phosphor dots.
  2. Electron beams.
  3. Focusing coils.
  4. Deflection coils.
  5. Anode connection.
  6. A mask for separating the beams for the red, green, and blue parts of the displayed image.
  7. Phosphor layer with red, green, and blue zones.
  8. Close-up of the phosphor-coated inner side.



The History
of

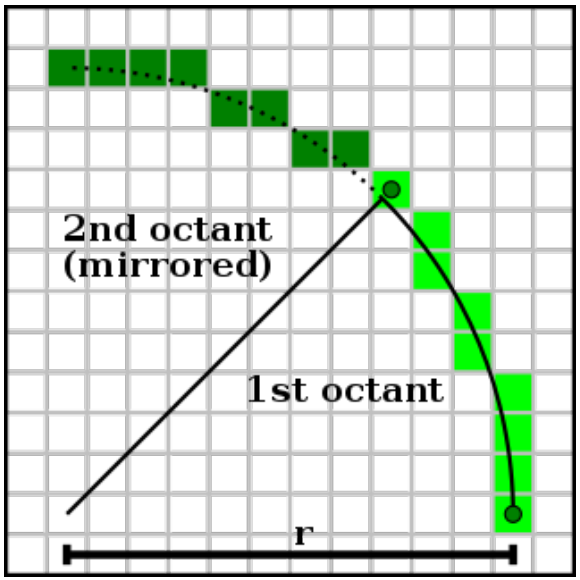
Computer Graphics



Line Drawing Algorithm

Module III: Computer Graphics

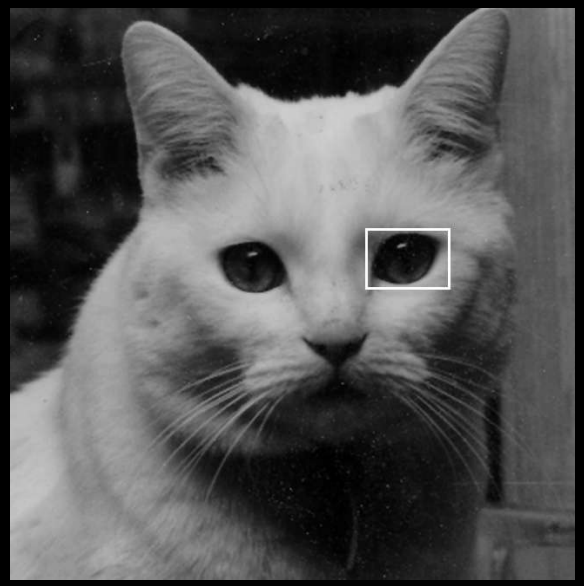
- Computer graphics in 1965, Jack Bresenham invented the “ideal” line-drawing algorithm.



Raster Graphics

Module III: Computer Graphics

- Computer graphics in 1970-1980:
  - Raster Graphics began to appear.
  - Image is produced as an array (the raster) of picture elements (pixels) in the frame buffer.

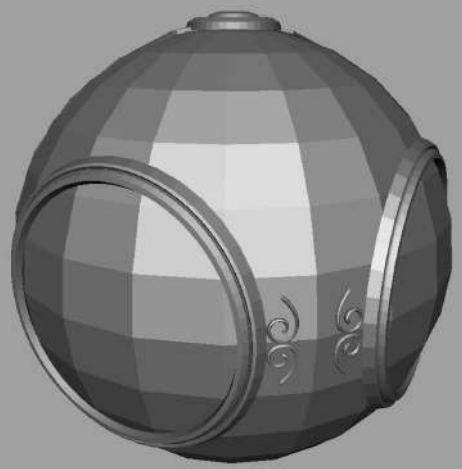




Raster Graphics

Module III: Computer Graphics

- Allows us to go from lines and wireframe images to filled polygons.
  - Low-polygon Model



PCs and Workstations

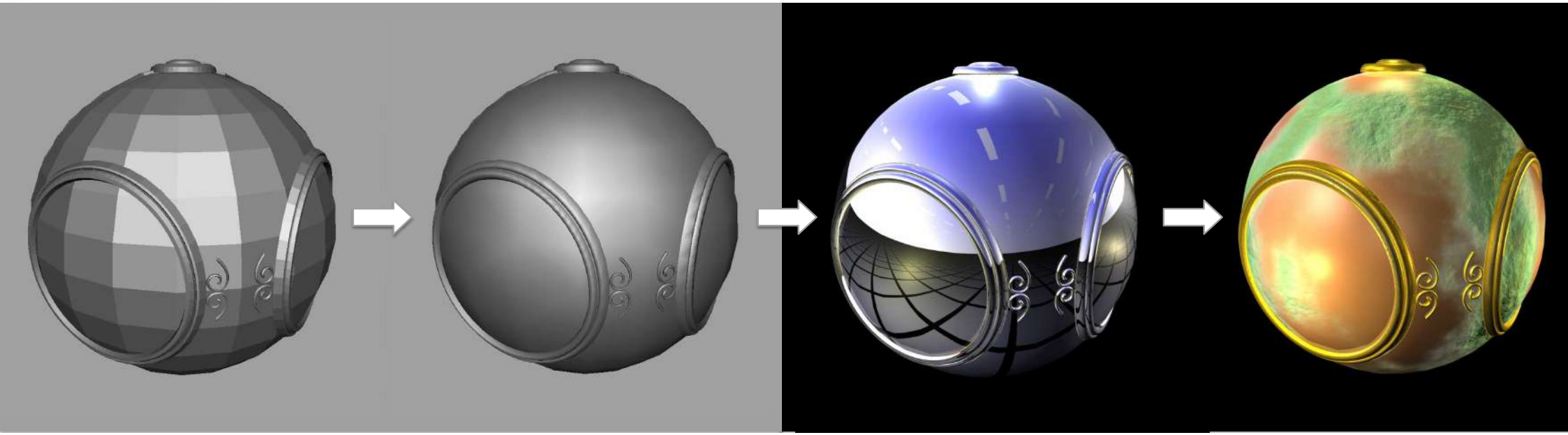
Module III: Computer Graphics

- Although we no longer make the distinction between workstations and PCs, historically, they evolved from different roots.
  - Early workstations were characterized by
    - Networked connection: client-server model
    - High level of interactivity.
  - Early PCs included a frame buffer as part of user memory.
    - Easy to change contents and create images (can copy pixels to user memory, but refresh very slowly).
    - The Apple II is the first graphics PC.
    - A 2D CAD package was released.

Realism Graphics

Module III: Computer Graphics

- Computer graphics in 1980-1990 came with realism.
  - Smooth Shading
  - Environment Mapping
  - Bump Mapping



OpenGL Application Programming Interface

Module III: Computer Graphics

- Computer graphics in 1990-2000 began to use the OpenGL Application Programming Interface (API).
- Completely computer-generated feature-length movies (Toy Story) are successful.
- New hardware capabilities:
  - Texture Mapping
  - Blending
  - Accumulation (consumer-grade graphics cards during the 1990s did not accelerate the Accumulation buffer in hardware),
  - Stencil buffer (an extra data buffer, Z-buffer and stencil buffer often share the same area in the RAM of the graphics hardware).

Computer Graphics 2000+

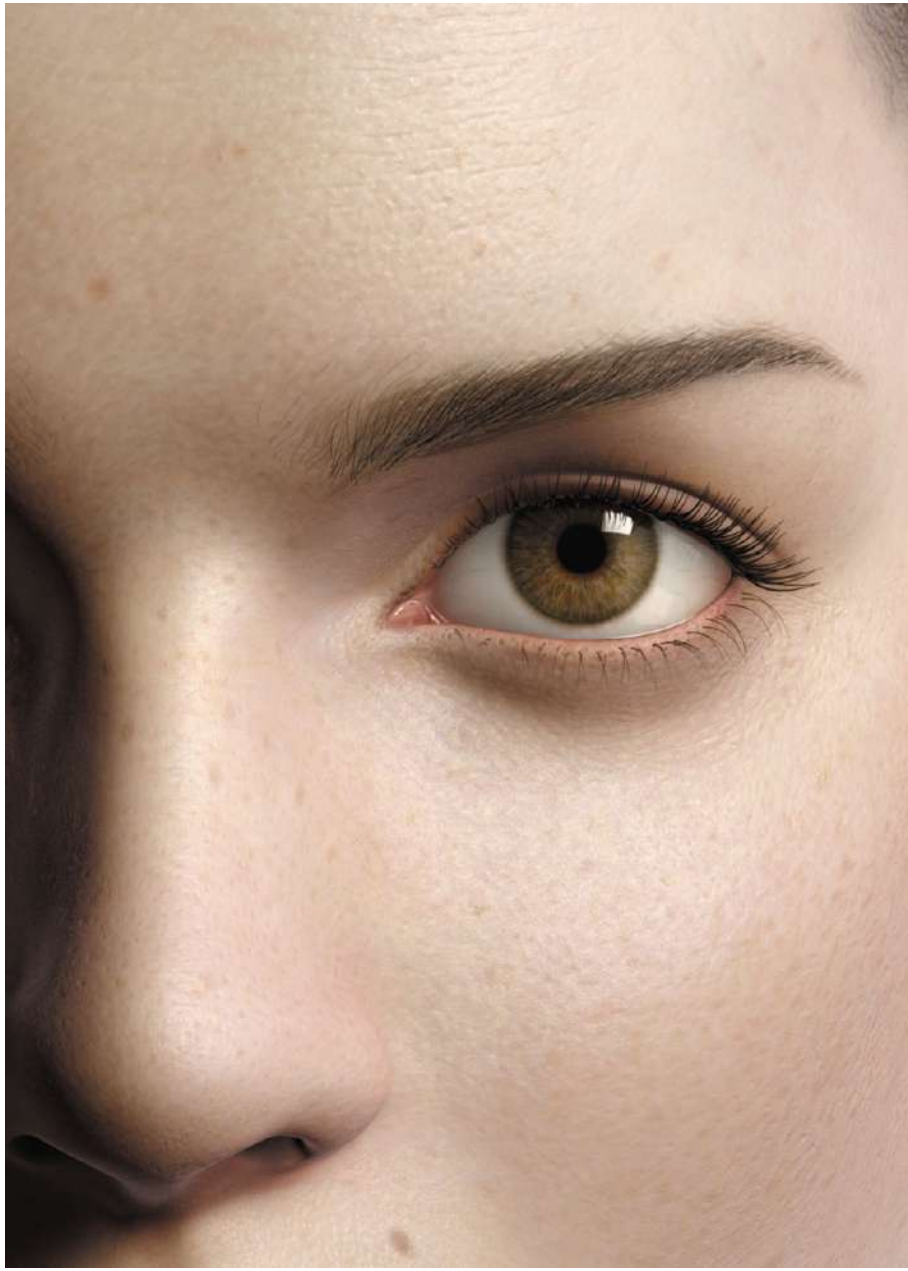
Module III: Computer Graphics

- Photorealism
- Graphics cards for PCs dominate the market:
  - NVidia
  - ATI
- Game boxes and game players determine the market's direction.
- Computer graphics routine in the movie industry
  - Maya
  - Lightwave
- Programmable pipelines

Water Effect

Module III: Computer Graphics

- The Final Fantasy movie was released in 2001.
- Rendering tears and water are now possible, but the movement of hair is still un-renderable.



Game Engine

Module III: Computer Graphics

- Doom's graphics engine for the game was released in 2003, but in 2019, we have Doomsday Engine on https://dengine.net
  - https://manual.dengine.net/guide/getting_started?redirect=1





Image Formation

Module III: Computer Graphics

- Fundamental imaging notions
- Physical basis for image formation
  - Light
  - Color
  - Perception
- Synthetic camera model
- Other models

Image Formation

Module III: Computer Graphics

- In computer graphics, we form images, which are generally two-dimensional using a process analogous to how images are formed by physical imaging systems
  - Cameras
  - Microscopes
  - Telescopes
  - Human visual system

Perception
in

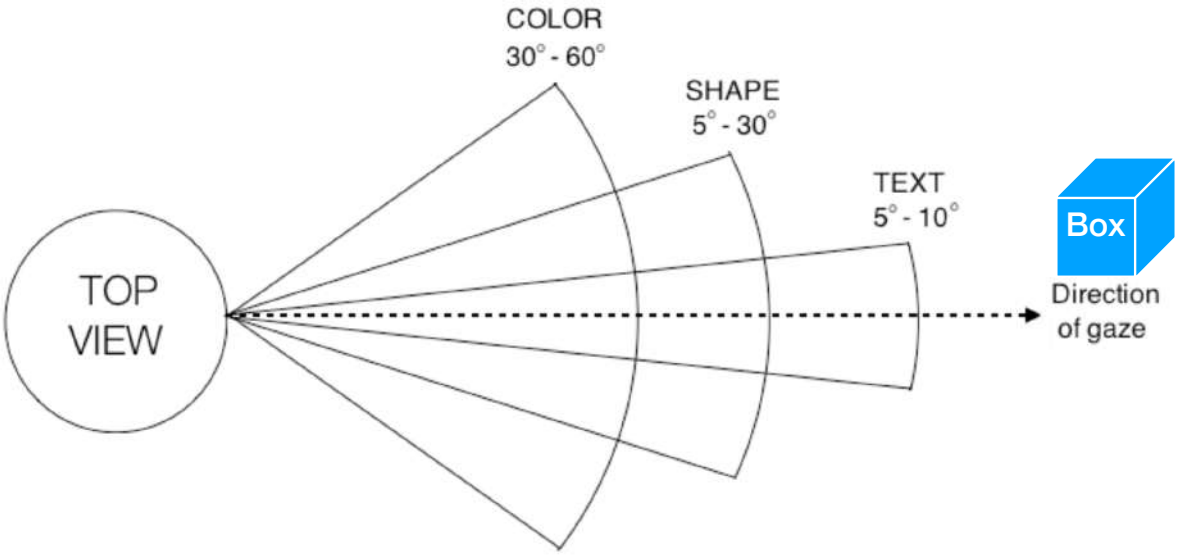
Color & Color Systems



Color

Module III: Color Perception

- The human visual system is capable of a lot of things.
- The top view of a human and the angles at which a human sees various things.

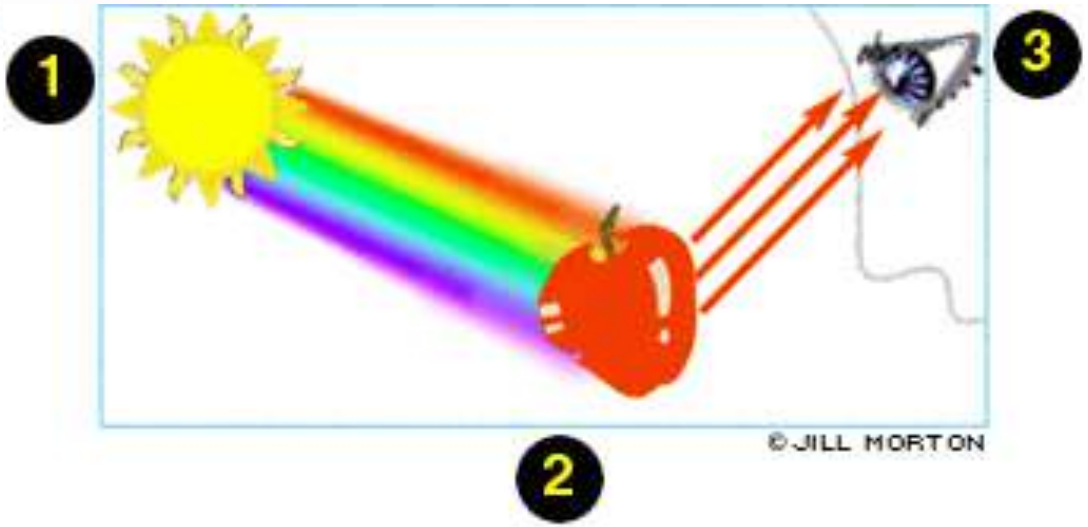


source: P. Joshi, D. M. Escriva and V. Godoy, "OpenCV by Example," Packt Publishing, Birmingham - Mumbai, 297 pages, ISBN 978-1-78528-094-8, Jan 2016.

Color

Module III: Color Perception

- Color originates in light.
- All the invisible colors of sunlight shine on the apple.
- The surface of the red apple absorbs all the colored light rays, except for those corresponding to red, and reflects this color to the human eye.



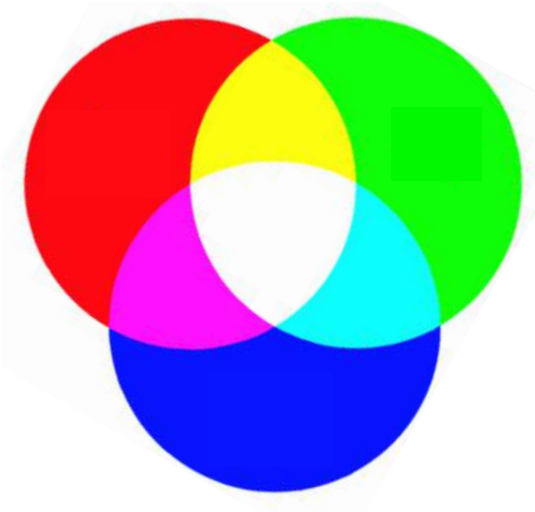
source: http://www.colormatters.com

- The human eye receives the reflected red light and sends a message to the brain.

RGB Color System

Module III: Color Perception

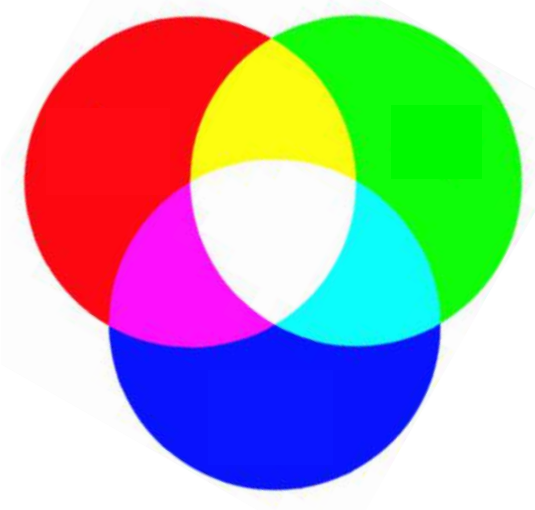
- The colors generated by light, such as the color system in a computer monitor, are an example to describe the RGB color system by using the Additive Color System/Theory, composed of Red, Green, and Blue (RGB).



RGB Color System

Module III: Color Perception

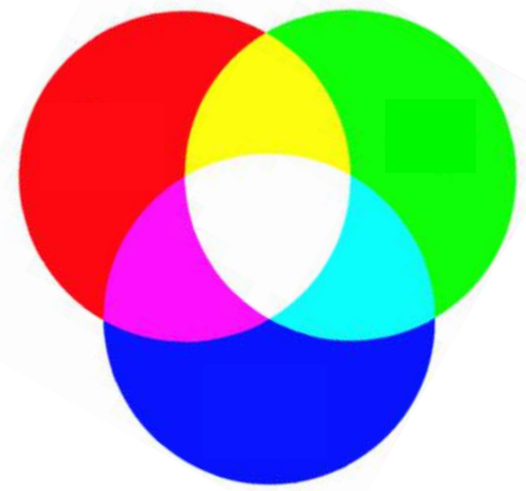
- This color system is applied only to the devices employing light, such as:
  - computer monitors
  - television sets.



RGB Color System

Module III: Color Perception

- Mixing of the color lights:
  - Red and Green light rays produce Yellow.
  - Blue and Green produce Cyan.
  - Red and Blue produce Magenta.
  - Red, Green, and Blue mix to create White (light).



Grayscale Color System

Module III: Color Perception

- Grayscale Color System
  - The intensity of light emitted from a surface, or called luminance, is a weighted sum of three corresponding linear intensity values (RGB) as in the equation below.

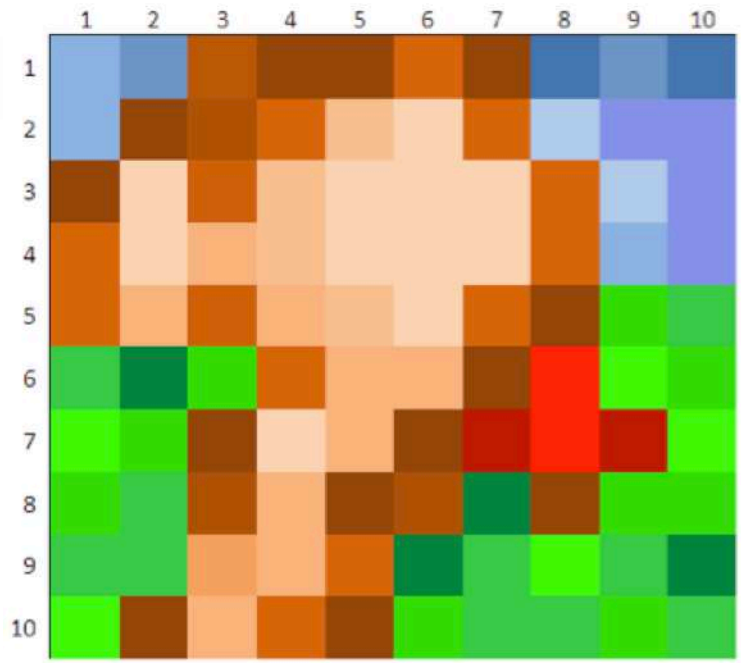
$$I = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

  - Each color channel has a 256-value color range (0-255) — in 8-bit color depth data.
  - 24-bit total color = 16.7 million colors.

Grayscale Color System

Module III: Color Perception

- Grayscale Color System
  - Described by a grayscale value and demonstrated as in the following figures.



(a) RGB color image.

	1	2	3	4	5	6	7	8	9	10
1	174	144	109	88	88	126	88	110	144	110
2	174	88	103	126	204	221	126	200	153	153
3	88	221	122	204	221	221	221	126	200	153
4	126	221	196	204	221	221	221	126	174	153
5	126	196	122	196	204	221	126	88	146	144
6	144	86	146	126	196	196	88	76	170	146
7	170	146	88	221	196	88	57	76	57	170
8	146	144	103	196	88	103	86	88	146	146
9	144	144	181	196	126	86	144	170	144	86
10	170	88	196	126	88	146	144	144	146	144

(b) Grayscale image with I values.

CMYK Color System

Module III: Color Perception

- CMYK Color System
  - Generally used in the printing/publishing industry using inks.
  - Composed of Cyan, Magenta, Yellow, and blacK.



source: http://www.colormatters.com

CMYK Color System

Module III: Color Perception

- CMYK Color System

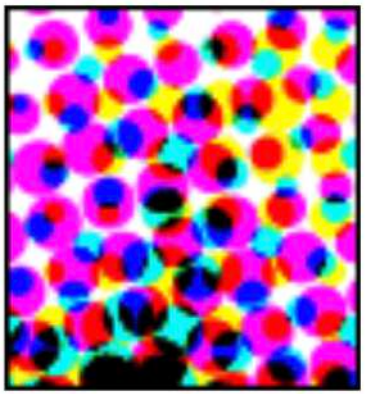


source: https://www.corporate3design.com/graphic-design-101-cmyk-or-rgb-color/

CMYK Color System

Module III: Color Perception

- CMYK Color System
  - Mixed result is Gray color.
  - The illustration, when look at a printed page with a magnifying glass, is demonstrated in below picture:

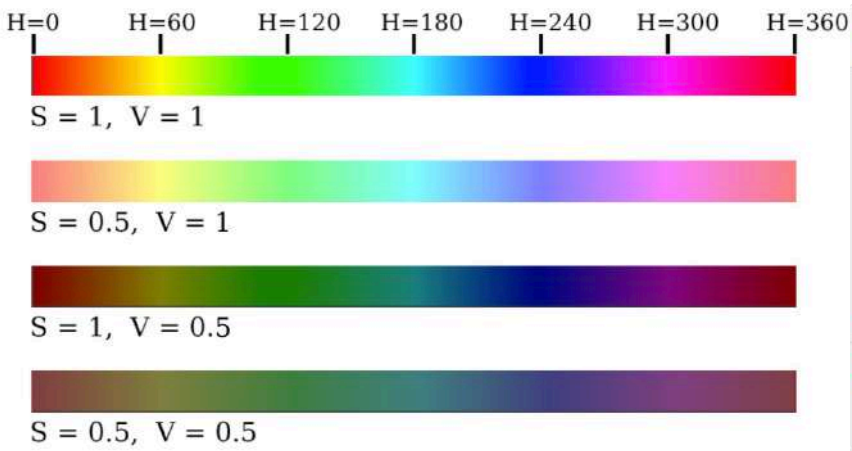


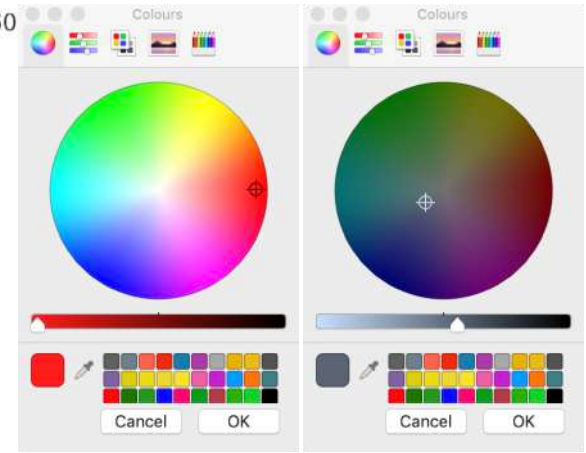
source: http://www.colormatters.com

HSV Color System

Module III: Color Perception

- RGB is sometimes criticized as being unintuitive.
  - It's not obvious to most people that yellow is made of a combination of red and green.
- The closely related color models HSV and HSL describe the same set of colors as RGB, but attempt to do it more intuitively.
  - HSV is sometimes called HSB, with the "V" or "B" standing for "brightness"

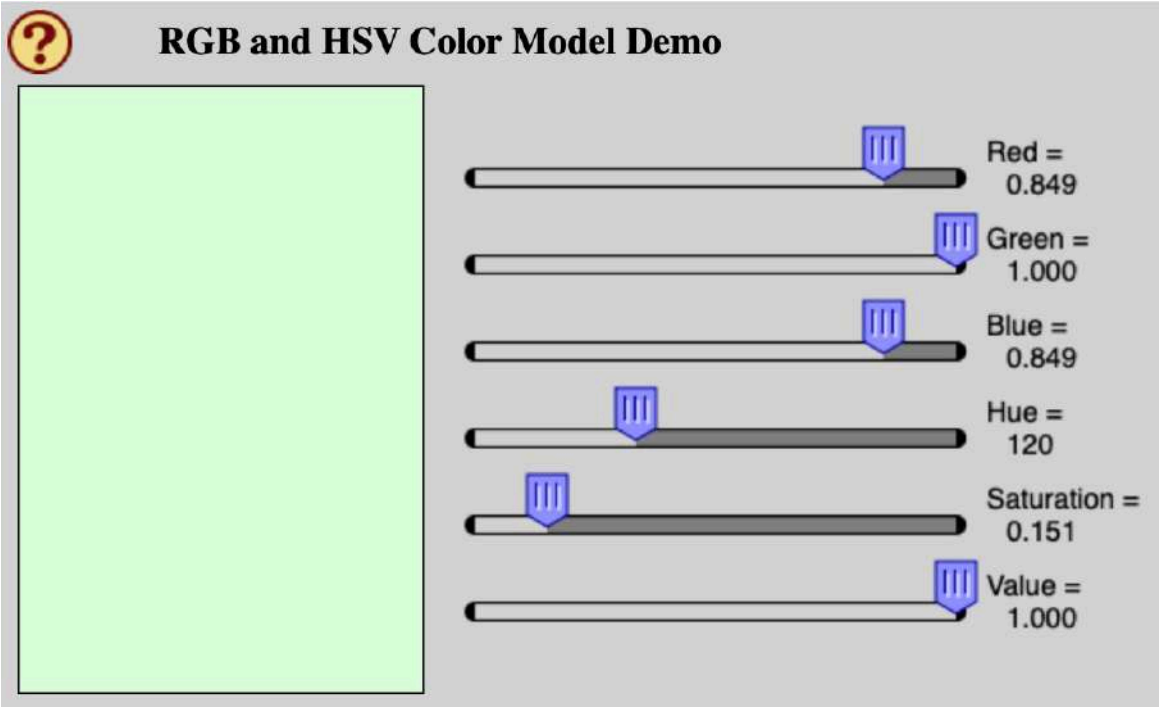




HSV Color System

Module III: Color Perception

- Try this: https://math.hws.edu/graphicsbook/demos/c2/rgb-hsv.html



Introduction
to

Pixels



Pixels

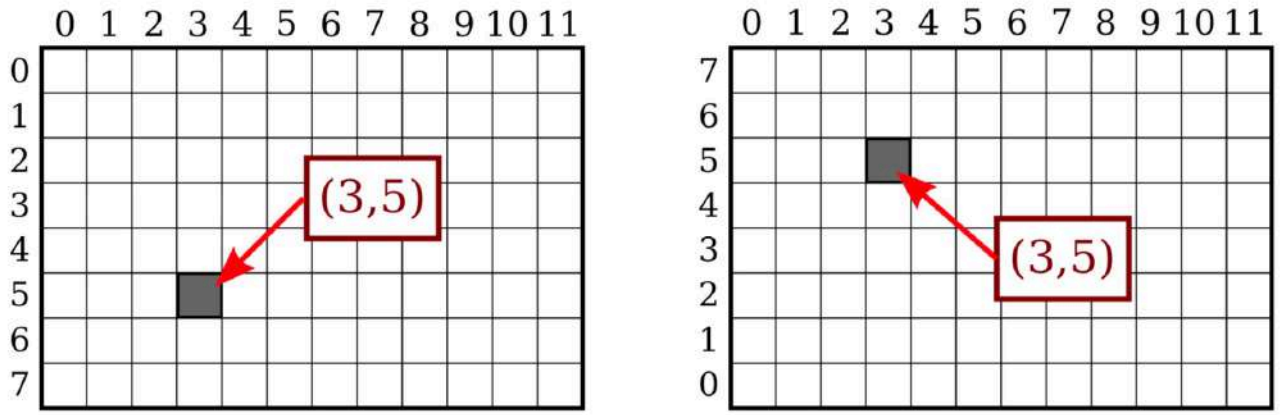
Module III: Pixels

- A point in 2D can be identified by a pair of numerical coordinates, and it is assigned a color — ‘pixel’.
- In terms of coordinates, a pixel can be identified by a pair of integers giving the column number and row number.

Pixel Coordinates

Module III: Pixels

- A digital image is made up of rows and columns of pixels.
- A pair of integers giving the column number and row number, for example, the pixel with coordinates (3, 5) would lie in column number 3 and row number 5.



12-by-8 pixel grids, shown with row and column numbers.
On the left, rows are numbered from top to bottom,
on the right, they are numberd bottom to top.

Algorithms
of

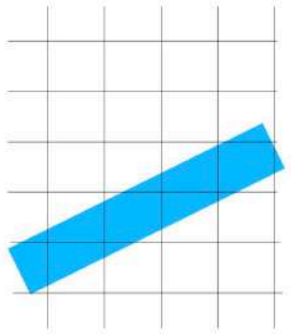
Output Primitives

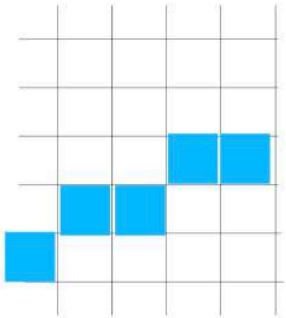


Output Primitives

Module III: Drawing I

- Graphic software and hardware provide subroutines to describe a scene in terms of basic geometric structures — called “output primitives”.
- Output primitives are combined to form complex structures.
- Simplest primitives
  - Point (pixel)
  - Line segment

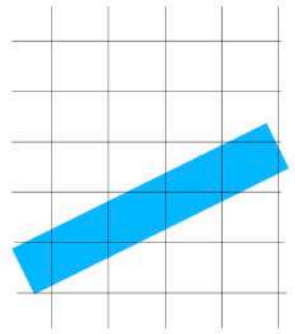


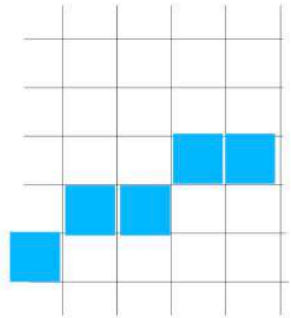


Scan Conversion

Module III: Drawing I

- Converting output primitives into frame buffer updates.
- Choose which pixels contain which intensity value.
- Constraints:
  - Straight lines should appear as straight lines.
  - Primitives should start and end accurately.
  - Primitives should have a consistent brightness along their length.
  - They should be drawn rapidly.





Line Drawing

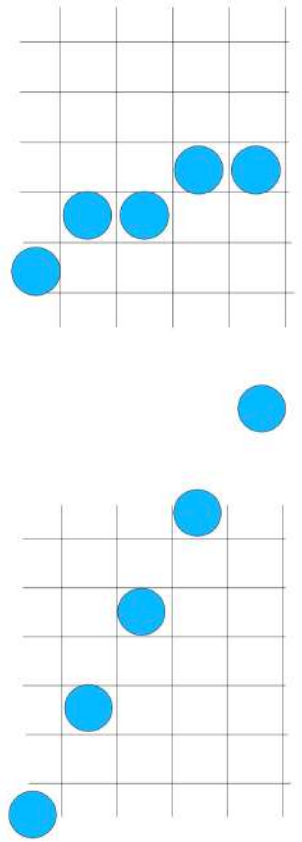
Module III: Drawing I

- Simple approach:
  - Sample a line at discrete positions at one coordinate from the start point to the end point.
  - Calculate the other coordinate value from the line equation.

$$y = m\,x + b \longrightarrow x = \frac{1}{m}y + \frac{b}{m}$$

$$m = \frac{y_{start} - y_{end}}{x_{start} - x_{end}} = \frac{\Delta y}{\Delta x}$$

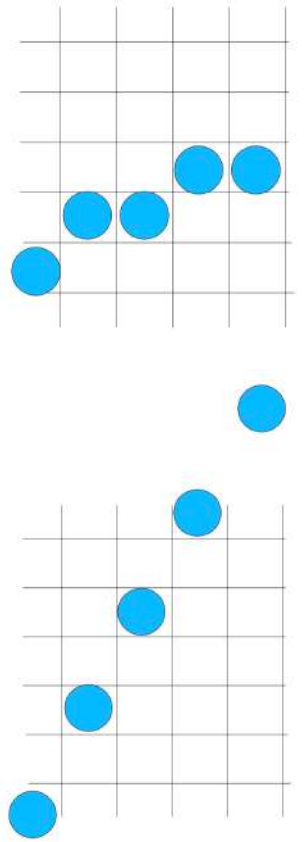
  - If $m > 1$, then increment y and find x.
  - Otherwise, increment x and find y.



Line Drawing Algorithms

Module III: Drawing I

- Digital Differential Analyzer (DDA) Line Drawing Algorithm
- Bresenham Line Drawing Algorithm
- Midpoint Line Drawing Algorithm



Digital Differential Analyzer (DDA)

Module III: Drawing I

- Procedure:

  Given-
  - Starting coordinates = (x_0, y_0)
  - Ending coordinates = (x_n, y_n)

- The points generation using the DDA Algorithm involves the following steps:

Digital Differential Analyzer (DDA)

Module III: Drawing I

- Step 1:

  Calculate Δx , Δy and m from the given input.

  These parameters are calculated as-

  - $\Delta x = x_n - x_0$
  - $\Delta y = y_n - y_0$
  - $m = \frac{\Delta y}{\Delta x}$

Digital Differential Analyzer (DDA)

Module III: Drawing I

- Step 2:

  Find the number of steps or points in between the starting and ending coordinates.
  - if $|\Delta x| \geq |\Delta y|$
    - steps Δx
  - else
    - steps Δy
  - then
    - $x\text{Inc} = \Delta x$ / steps
    - $y\text{Inc} = \Delta y$ / steps

Digital Differential Analyzer (DDA)

Module III: Drawing I

- Step 3:

  Suppose the current point is (x_k, y_k) and the next point is (x_{k+1}, y_{k+1}), when $k = 0, 1, 2, \ldots$, find the next point by:

  - $y_{k+1} = y_k + y\text{Inc}$
  - $x_{k+1} = x_k + x\text{Inc}$

Digital Differential Analyzer (DDA)

Module III: Drawing I

- Step 4:

  Keep repeating Step 3 until the endpoint is reached or the number of generated new points (including the starting and ending points) equals the steps's count.

Digital Differential Analyzer (DDA)

Module III: Drawing I

- Practice Problems based on DDA Algorithm:

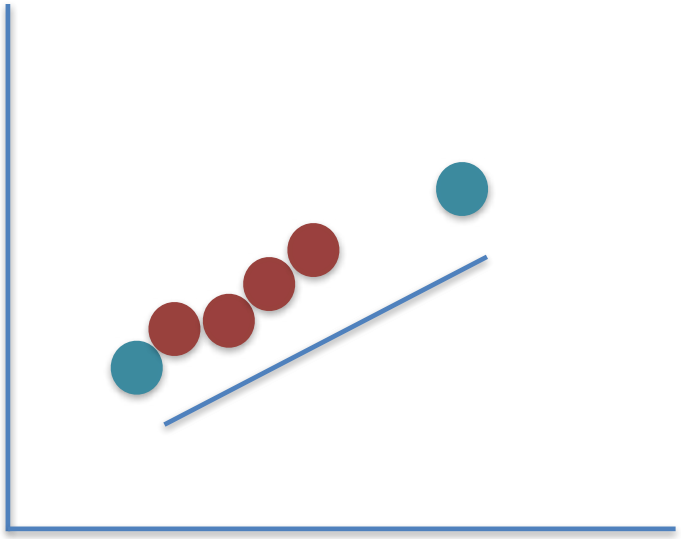
  Problem 1-

  Calculate the points between the starting point (5, 6) and the ending point (8, 12).

  Solution:

  Given-

  - Starting coordinates: $(x_0, y_0) = (5, 6)$
  - Ending coordinates: $(x_n, y_n) = (8, 12)$



Digital Differential Analyzer (DDA)

Module III: Drawing I

- Step 1:

  Calculate Δx , Δy and m from the given input.

  - $\Delta x = x_n - x_0 = x_n - x_0 = 8 - 5 = 3$
  - $\Delta y = y_n - y_0 = 12 - 6 = 6$
  - $m = \frac{\Delta y}{\Delta x} = 6/3 = 2$

- Step 2:

  Calculate the number of steps.

  As $|\Delta x| < |\Delta y| = 3 < 6$, so the number of steps $= \Delta y = 6$

Digital Differential Analyzer (DDA)

Module III: Drawing I

- Step 3:

  As number of steps = Δy = 6, $x\text{Inc}$ = 3/6, and $y\text{Inc}$ = 6/6

  So, now Step 3 is executed until Step 4 is satisfied.

x_k	y_k	x_{k+i}	y_{k+i}	round(x_{k+i}, y_{k+i})
5	6	5.5	7	(6, 7)
		6	8	(6, 8)
		6.5	9	(7, 9)
		7	10	(7, 10)
		7.5	11	(8, 11)
		8	12	(8, 12)

Digital Differential Analyzer (DDA)

Module III: Drawing I

- Advantages
  - It is a simple algorithm.
  - It is easy to implement.
  - It avoids using the multiplication operation, which is costly in terms of time complexity.
- Disadvantages
  - There is an extra overhead of using the round off( ) function.
  - Using the round off( ) function increases the algorithm's time complexity.
  - Resulted lines are not smooth because of rounding off( ).
  - The points generated by this algorithm are not accurate.

Bresenham Line Drawing

Module III: Drawing I

- Procedure:

  Given-
  - Starting coordinates = (x_0, y_0)
  - Ending coordinates = (x_n, y_n)

- The points generation using the Bresenham Line Drawing Algorithm involves the following steps:

Bresenham Line Drawing

Module III: Drawing I

- Step 1:

  Calculate Δx and Δy from the given input. These parameters are calculated as-

  - $\Delta x = x_n - x_0$
  - $\Delta y = y_n - y_0$

Bresenham Line Drawing

Module III: Drawing I

- Step 2:

  Calculate the decision parameter p_k, when $k = 0, 1, 2, \ldots$.

  $$p_k = 2\Delta y - \Delta x$$

Bresenham Line Drawing

Module III: Drawing I

- Step 3:

  Suppose the current point is (x_k, y_k) and the next point is (x_{k+1}, y_{k+1}), when $k = 0, 1, 2, \ldots$.
  - Find the next point depending on the value of decision parameter p_k.
  - Follow the below two cases.
  - Case 1: if $p_k < 0$

$$p_{k+1} = p_k + 2\Delta y$$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k$$

Bresenham Line Drawing

Module III: Drawing I

- Step 3:

  Suppose the current point is (x_k, y_k) and the next point is (x_{k+1}, y_{k+1}), when $k = 0, 1, 2, \ldots$.
  - Find the next point depending on the value of decision parameter p_k.
  - Follow the below two cases.
  - Case 2: if $p_k \geqslant 0$

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + 1$$

Bresenham Line Drawing

Module III: Drawing I

- Step 4:

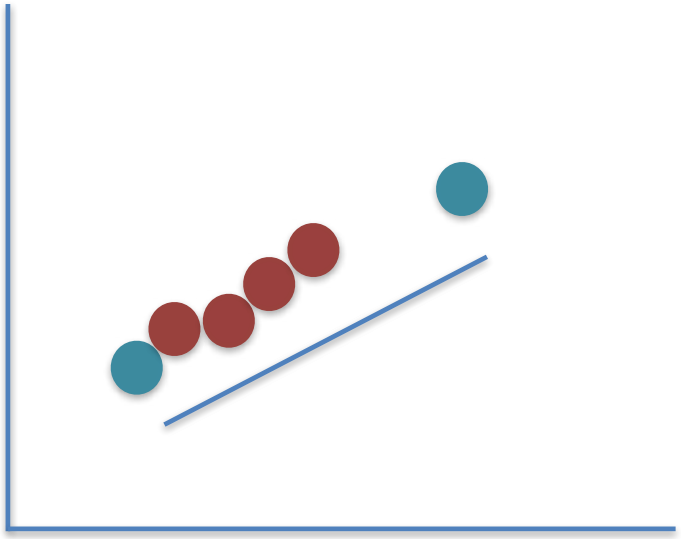
  Keep repeating Step 3 until the end point is reached or the number of iterations is equal to $(\Delta x - 1)$ times.

Bresenham Line Drawing

Module III: Drawing I

- Practice Problems based on the Bresenham Line Drawing Algorithm:
  Problem 1-
  Calculate the points between the starting point (9, 18) and the ending point (14, 22).

  Solution:
  Given-
  - Starting coordinates: $(x_0, y_0) = (9, 18)$
  - Ending coordinates: $(x_n, y_n) = (14, 22)$



Bresenham Line Drawing

Module III: Drawing I

- Step 1:

  Calculate Δx and Δy from the given input.

  - $\Delta x = x_n - x_0 = 14 - 9 = 5$
  - $\Delta y = y_n - y_0 = 22 - 18 = 4$

- Step 2:

  Calculate the decision parameter p_k.

$$p_k = 2\Delta y - \Delta x$$

$$p_k = 2(4) - 5$$

$$p_k = 3$$

Bresenham Line Drawing

Module III: Drawing I

- Step 3:

  As $p_k > 0$, so Case 2 is satisfied.

  So, $p_{k+1} = p_k + 2\Delta y - 2\Delta x = 3 + 2(4) - 2(5) = 1$

  $x_{k+1} = x_k + 1 = 9 + 1 = 10$

  $y_{k+1} = y_k + 1 = 18 + 1 = 19$

  Similarly, Step 3 is executed until the end point is reached or number of iterations is equal to 4 times.

  Number of iteration $= \Delta x - 1 = 5 - 1 = 4$

p_k	p_{k+1}	x_{k+1}	y_{k+1}
		9	18
3	1	10	19
1	-1	11	20
-1	7	12	20
7	5	13	21
5	3	14	22

Bresenham Line Drawing

Module III: Drawing I

- Advantages
  - It is easy to implement.
  - It is fast and incremental.
  - It executes fast but less faster than the DDA Algorithm.
  - The points generated by this algorithm are more accurate than the DDA Algorithm.
  - It uses fixed points only.

Bresenham Line Drawing

Module III: Drawing I

- Disadvantages
  - Though it improves the accuracy of generated points but still the resulting line is still not smooth.
  - This algorithm is for basic line drawing.
  - It can not handle diminishing jaggedness.

Midpoint Line Drawing Algorithm

Module III: Drawing I

- Procedure:

  Given-
  - Starting coordinates = (x_0, y_0)
  - Ending coordinates = (x_n, y_n)

- The points generation using the Midpoint Line Drawing Algorithm involves the following steps:

Midpoint Line Drawing

Module III: Drawing I

- Step 1:

  Calculate Δx and Δy from the given input. These parameters are calculated as-

  - $\Delta x = x_n - x_0$
  - $\Delta y = y_n - y_0$

Midpoint Line Drawing

Module III: Drawing I

- Step 2:

  Calculate the value of the initial decision parameter and ΔD.

  $$D_0 = 2\Delta y - \Delta x$$

  $$\Delta D = 2(\Delta y - \Delta x)$$

Midpoint Line Drawing

Module III: Drawing I

- Step 3:

  The decision whether to increment the x or y coordinate depends upon the following values of D_k, when $k = 0, 1, 2, \ldots$.

  - Case 1: if $D_k < 0$

    $$x_{k+1} = x_k + 1$$

    $$y_{k+1} = y_k$$

    $$D_{k+1} = D_k + 2\Delta y$$

Midpoint Line Drawing

Module III: Drawing I

- Step 3:

  The decision whether to increment the x or y coordinate depends upon the following values of D_k, when $k = 0, 1, 2, \ldots$.

  - Case 2: if $D_k \geqslant 0$

    $$x_{k+1} = x_k + 1$$

    $$y_{k+1} = y_k + 1$$

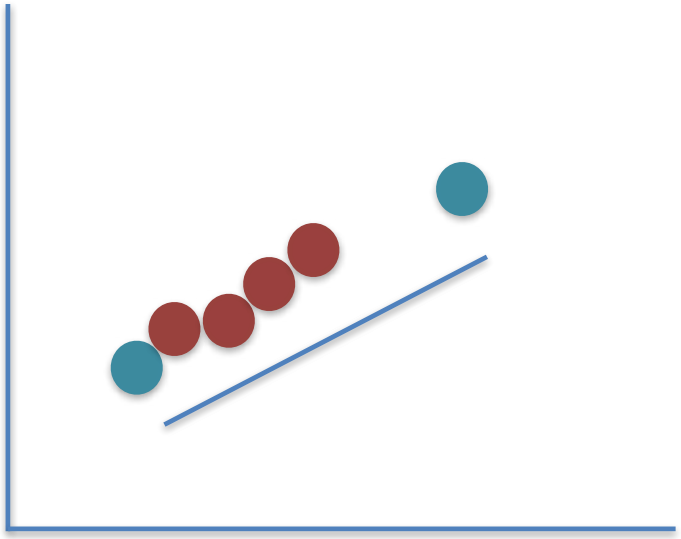
    $$D_{k+1} = D_k + 2\Delta D$$

Midpoint Line Drawing

Module III: Drawing I

- Practice Problems based on the Bresenham Line Drawing Algorithm:
  Problem 1-
  Calculate the points between the starting point (20, 10) and the ending point (30, 18).

  Solution:
  Given-
  - Starting coordinates: $(x_0, y_0) = (20, 10)$
  - Ending coordinates: $(x_n, y_n) = (30, 18)$



Midpoint Line Drawing

Module III: Drawing I

- Step 1:

  Calculate Δx and Δy from the given input.

  - $\Delta x = x_n - x_0 = 30 - 20 = 10$
  - $\Delta y = y_n - y_0 = 18 - 10 = 8$

- Step 2:

  Calculate D_0 and ΔD as-

$$D_0 = 2\Delta y - \Delta x = 2(8) - 10 = 6$$

$$\Delta D = 2(\Delta y - \Delta x) = 2(8 - 10) = -4$$

Midpoint Line Drawing

Module III: Drawing I

- Step 3:

  As $D_k > 0$, so Case 2 is satisfied.

  So, $x_{k+1} = x_k + 1 = 20 + 1 = 21$

  $y_{k+1} = y_k + 1 = 10 + 1 = 11$

  $D_{k+1} = D_k + \Delta D = 6 + (-4) = 2$

  Similarly, Step 3 is executed until the endpoint is reached.

$D_{initial}$	D_{new}	x_{k+1}	y_{k+1}
		20	10
6	2	21	11
2	-2	22	12
-2	14	23	12
14	10	24	13
10	6	25	14
6	2	26	15
2	-2	27	16
-2	14	28	16
14	10	29	17
10		30	18

Midpoint Line Drawing

Module III: Drawing I

- Advantages
  - The accuracy of finding points is a key feature of this algorithm.
  - It is simple to implement.
  - It uses basic arithmetic operations.
  - It takes less time for computation.
  - The resulting line is smoother as compared to other line drawing algorithms.

Midpoint Line Drawing

Module III: Drawing I

- Disadvantages
  - This algorithm may not be an ideal choice for complex graphics and images.
  - In terms of the accuracy of finding points, improvement is still needed.
  - There is no remarkable improvement made by this algorithm.

Computer Graphics: Python Programming on Colab

Colab: User-defined Functions

Module III: Computer Graphics

Step 1: Import IPython and output

```
import IPython
from google.colab import output

#… continue
```

- The code snippet in Google Colab notebooks enables interactive HTML/JavaScript visualizations.
  - Imports the IPython library, which provides tools for interactive computing in Jupyter/Colab notebooks.
  - The IPython.display module includes methods like HTML(), display(), and clear_output() to render rich content (HTML, images, videos, etc.) directly in notebook cells.
  - In the code (you will see later), IPython.display.HTML(html_code) converts the HTML string into a renderable object.

Colab: User-defined Functions

Module III: Computer Graphics

Step 1: Import IPython and output

```
import IPython
from google.colab import output

#… continue
```

- The code snippet in Google Colab notebooks enables interactive HTML/JavaScript visualizations.
    - Imports Colab-specific utilities for controlling notebook output.
    - output.enable_custom_widget_manager(): Enables custom interactive widgets (e.g., IPyWidgets).
    - output.clear(): Clears the output of a cell (useful for dynamic updates).
    - In the snippet, output isn't directly used, but it's often paired with IPython for advanced interactivity (e.g., clearing outputs before re-rendering).

Colab: User-defined Functions

Module III: Computer Graphics

Step 2: Create embedded html code

```
#… continue

html_code = """
<!DOCTYPE html>
<html>

#… continue
```

- Declares the document type as HTML5 (modern web standard) by <!DOCTYPE html>.
    - This ensures browsers/rendering engines interpret the content correctly.
- Everything inside the tag <html> defines the webpage's structure.
    - This ensures browsers/rendering engines interpret the content correctly.
    - Contains two main parts:
        - <head>: Metadata (title, styles, scripts).
        - <body>: Visible content (canvas, interactive elements).

Colab: User-defined Functions

Module III: Computer Graphics

Step 2: Create embedded html code for <head></head>

```
#… continue

<head>
<title>Interactive Bresenham's Line Algorithm</title>
<style>
  body {
    margin: 0;
    overflow: hidden;
  }
  canvas {
    border: 1px solid black;
    cursor: crosshair;
  }
</style>
</head>

#… continue
```

- <title>: Sets the browser tab title (not visible in Colab).

Colab: User-defined Functions

Module III: Computer Graphics

Step 2: Create embedded html code for <head></head>

```
#… continue

<head>
<title>Interactive Bresenham's Line Algorithm</title>
<style>
  body {
    margin: 0;
    overflow: hidden;
  }
  canvas {
    border: 1px solid black;
    cursor: crosshair;
  }
</style>
</head>

#… continue
```

- The styling:
  - creates a clean, focused drawing environment.
  - removes all distractions (margins, scrollbars).
  - provides clear visual feedback about where you can draw.
  - The crosshair cursor reinforces that this is a precision drawing tool.

Colab: User-defined Functions

Module III: Computer Graphics

Step 2: Create embedded html code for <head></head>

```
#… continue

<head>
<title>Interactive Bresenham's Line Algorithm</title>
<style>
  body {
    margin: 0;
    overflow: hidden;
  }
  canvas {
    border: 1px solid black;
    cursor: crosshair;
  }
</style>
</head>

#… continue
```

- margin: 0;
  - Removes all default margin space around the body of the webpage—browsers add default margins (usually 8px) around webpages.
  - This would create unwanted space around our canvas.
  - The canvas will touch the very edges of the browser window.

Colab: User-defined Functions

Module III: Computer Graphics

Step 2: Create embedded html code for <head></head>

```
#… continue

<head>
<title>Interactive Bresenham's Line Algorithm</title>
<style>
  body {
    margin: 0;
    overflow: hidden;
  }
  canvas {
    border: 1px solid black;
    cursor: crosshair;
  }
</style>
</head>

#… continue
```

- overflow: hidden;
  - Prevents scrollbars from appearing.
    - If our content were larger than the window, this would stop scrollbars from appearing.
    - Ensures the canvas is the only visible element.
  - Even if you drag/resize the window, no scrollbars will show.

Colab: User-defined Functions

Module III: Computer Graphics

Step 2: Create embedded html code for <head></head>

```
#… continue

<head>
<title>Interactive Bresenham's Line Algorithm</title>
<style>
  body {
    margin: 0;
    overflow: hidden;
  }
  canvas {
    border: 1px solid black;
    cursor: crosshair;
  }
</style>
</head>

#… continue
```

- border: 1px solid black;
  - Adds a thin black border around the canvas.
  - Makes the canvas edges visible.
  - Provides a visual definition for the drawing area.
  - Creates a black rectangular frame around your drawing space.

Colab: User-defined Functions

Module III: Computer Graphics

Step 2: Create embedded html code for <head></head>

```
#… continue

<head>
<title>Interactive Bresenham's Line Algorithm</title>
<style>
  body {
    margin: 0;
    overflow: hidden;
  }
  canvas {
    border: 1px solid black;
    cursor: crosshair;
  }
</style>
</head>

#… continue
```

- cursor: crosshair;
  - Changes the mouse cursor to a crosshair (+) when over the canvas.
  - Indicates this is an interactive drawing area.
  - The crosshair helps with precision when selecting points.
  - Our normal arrow cursor becomes a precision crosshair when over the canvas.

Colab: User-defined Functions

Module III: Computer Graphics

Step 2: Create embedded html code for <head></head>

```
#… continue

<head>
<title>Interactive Bresenham's Line Algorithm</title>
<style>
  body {
    margin: 0;
    overflow: hidden;
  }
  canvas {
    border: 1px solid black;
    cursor: crosshair;
  }
</style>
</head>

#… continue
```

- Combined Effects:
  - The webpage will have:
    - No margins or padding.
    - No scrollbars (even if you resize the window).

Colab: User-defined Functions

Module III: Computer Graphics

Step 2: Create embedded html code for <head></head>

```
#… continue

<head>
<title>Interactive Bresenham's Line Algorithm</title>
<style>
  body {
    margin: 0;
    overflow: hidden;
  }
  canvas {
    border: 1px solid black;
    cursor: crosshair;
  }
</style>
</head>

#… continue
```

- Combined Effects:
  - The canvas will have:
    - A visible black border.
    - A crosshair cursor for precise pointing.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- These lines set up the canvas by:
  - Finding the canvas element in the HTML
  - Getting its drawing tools
  - Setting its size
  - Preparing a variable to track user clicks

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <canvas>: A drawable area (used for rendering lines via JavaScript).
- <script>: Contains JavaScript to:
  - Handle mouse clicks.
  - Implement Bresenham's line-drawing algorithm.
  - Animate the line pixel-by-pixel.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- The <canvas id="myCanvas"></canvas> element is a fundamental HTML component that creates a drawable region on a webpage. Here's a detailed explanation:
  - An HTML5 element that provides a blank rectangular area where you can draw graphics using JavaScript.
  - Acts like a blank artist's canvas that you can programmatically paint on.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <script>: Step-by-Step
  - document: refers to the HTML document (web page) currently loaded in the browser.
  - getElementById('myCanvas'): a method that searches the document for an HTML element with the id="myCanvas" and returns the first matching element (since IDs must be unique in HTML).

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <script>: Step-by-Step
  - id="myCanvas": gives the canvas a unique identifier ("myCanvas").
  - Allows JavaScript to specifically target this element using document.getElementById()
  - Important when you have multiple canvas elements on a page

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <script>: Step-by-Step
  - const canvas = ...
    - stores the found element in a variable named canvas for later use.
    - const means this variable cannot be reassigned (though the canvas properties can be modified).

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <script>: Step-by-Step
  - const canvas = ...
    - gets a reference to the element defined earlier in HTML, <canvas id=“myCanvas”></canvas>, which allows JavaScript to interact with the canvas, draw lines, handle clicks, etc.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <script>: Step-by-Step
  - const ctx = ...
    - gets the drawing tools/interface for the canvas.
    - The canvas itself is just a blank space - we need this “context” to draw on it.
    - The getContext('2d') gives us tools for 2D drawing (as opposed to 3D).

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <script>: Step-by-Step
  - const ctx = … we can draw using methods like:
    - ctx.fillRect() - Draws filled rectangles
    - ctx.strokeRect() - Draws rectangular outlines
    - ctx.beginPath(), ctx.lineTo(), ctx.stroke() - Draws lines

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <script>: Step-by-Step
  - canvas.width = 500; and canvas.height = 300;
  - sets the canvas size to 500 pixels wide and 300 pixels tall.
  - This differs from setting size via CSS - here is for setting the actual drawing dimensions.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <script>: Step-by-Step
  - let startPoint = null;
    - creates a variable to remember where the user first clicked.
    - For the line-drawing interaction, where you click two points.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code for <body><canvas>…</canvas></body>

```
#… continue

<body>

<canvas id="myCanvas"></canvas>

<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

canvas.width = 500;
canvas.height = 300;

let startPoint = null;

#… continue
```

- <script>: Step-by-Step
  - let startPoint = null;
    - starts as null (no point selected yet).
    - After the first click, it stores the coordinates.
    - After the second click, it draws the line from those coordinates, then resets to null.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

canvas.addEventListener('click', (event) => {
  const rect = canvas.getBoundingClientRect();
  const x = event.clientX - rect.left;
  const y = event.clientY - rect.top;

  if (!startPoint) {
    startPoint = { x, y };
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.fillRect(x - 2, y - 2, 4, 4); // Mark the start point
  } else {
    bresenhamLine(startPoint.x, startPoint.y, x, y);
    startPoint = null;
  }
});

#… continue
```

- Event Listener Setup

  canvas.addEventListener('click', (event) => {...})
- Listens for click events on the canvas element

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

canvas.addEventListener('click', (event) => {
  const rect = canvas.getBoundingClientRect();
  const x = event.clientX - rect.left;
  const y = event.clientY - rect.top;

  if (!startPoint) {
    startPoint = { x, y };
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.fillRect(x - 2, y - 2, 4, 4); // Mark the start point
  } else {
    bresenhamLine(startPoint.x, startPoint.y, x, y);
    startPoint = null;
  }
});

#… continue
```

- Getting Click Position:
  - Calculates the exact click position relative to the canvas.
  - clientX and clientY give window-relative coordinates.
  - Subtracting rect.left/rect.top converts to canvas-relative coordinates.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

canvas.addEventListener('click', (event) => {
  const rect = canvas.getBoundingClientRect();
  const x = event.clientX - rect.left;
  const y = event.clientY - rect.top;

  if (!startPoint) {
    startPoint = { x, y };
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.fillRect(x - 2, y - 2, 4, 4); // Mark the start point
  } else {
    bresenhamLine(startPoint.x, startPoint.y, x, y);
    startPoint = null;
  }
});

#… continue
```

- Two-Click Logic:
  - Uses the startPoint variable to track the state.
  - First click (when startPoint is null):
    - Draws a 4x4 square to mark the starting point.
    - Clears any previous drawings.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

canvas.addEventListener('click', (event) => {
  const rect = canvas.getBoundingClientRect();
  const x = event.clientX - rect.left;
  const y = event.clientY - rect.top;

  if (!startPoint) {
    startPoint = { x, y };
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.fillRect(x - 2, y - 2, 4, 4); // Mark the start point
  } else {
    bresenhamLine(startPoint.x, startPoint.y, x, y);
    startPoint = null;
  }
});

#… continue
```

- Two-Click Logic:
  - Uses the startPoint variable to track the state.
  - Second click (when startPoint exists):
    - Calls the line-drawing function between the stored start point and the new click
    - Resets the startPoint for the next line

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

function bresenhamLine(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    function draw() {
        ctx.fillRect(x0, y0, 2, 2);
        if ((x0 === x1) && (y0 === y1)) return;
        let e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
        requestAnimationFrame(draw);
    }
    draw();
}

#… continue
```

- Key Algorithm Features:
  - Integer-only arithmetic - No floating-point math needed.
  - Pixel-perfect lines - Always chooses the closest pixels to the ideal line.
  - Optimized movement - Only makes necessary moves in x/y directions.
  - Animated visualization - Shows the line being drawn pixel-by-pixel.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

function bresenhamLine(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    function draw() {
        ctx.fillRect(x0, y0, 2, 2);
        if ((x0 === x1) && (y0 === y1)) return;
        let e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
        requestAnimationFrame(draw);
    }
    draw();
}

#… continue
```

- Function Setup
  - Takes start point (x0,y0) and end point (x1,y1) coordinates.
  - Will draw a line between these points on the canvas.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

function bresenhamLine(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    function draw() {
        ctx.fillRect(x0, y0, 2, 2);
        if ((x0 === x1) && (y0 === y1)) return;
        let e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
        requestAnimationFrame(draw);
    }
    draw();
}

#… continue
```

- Initial Calculations
- dx, dy: Absolute distances between points
- sx, sy: Direction of movement (1 or -1)
- err: Decision parameter for when to move in the y-direction.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

function bresenhamLine(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    function draw() {
        ctx.fillRect(x0, y0, 2, 2);
        if ((x0 === x1) && (y0 === y1)) return;
        let e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
        requestAnimationFrame(draw);
    }
    draw();
}

#… continue
```

- Recursive Drawing Function
- Draws a 2x2 pixel square at current (x0,y0).

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

function bresenhamLine(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    function draw() {
        ctx.fillRect(x0, y0, 2, 2);
        if ((x0 === x1) && (y0 === y1)) return;
        let e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
        requestAnimationFrame(draw);
    }
    draw();
}

#… continue
```

- Termination Check
  - Stops recursion when we reach the endpoint.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

function bresenhamLine(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    function draw() {
        ctx.fillRect(x0, y0, 2, 2);
        if ((x0 === x1) && (y0 === y1)) return;
        let e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
        requestAnimationFrame(draw);
    }
    draw();
}

#… continue
```

- Error Calculation and Movement
  - Adjusts the error term and moves either horizontally, vertically, or diagonally.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

function bresenhamLine(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    function draw() {
        ctx.fillRect(x0, y0, 2, 2);
        if ((x0 === x1) && (y0 === y1)) return;
        let e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
        requestAnimationFrame(draw);
    }
    draw();
}

#… continue
```

- Animation Loop
  - Recursively calls draw() for smooth animation.
  - Browser-optimized animation timing (≈60fps).
  - requestAnimationFrame(draw) is a JavaScript method that tells the browser to call a specified function (draw in this case) before the next repaint of the webpage.
    - It's the modern, optimized way to create smooth animations in browsers.

Colab: User-defined Functions

Module III: Computer Graphics

Step 3: Create embedded html code …

```
#… continue

function bresenhamLine(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    function draw() {
        ctx.fillRect(x0, y0, 2, 2);
        if ((x0 === x1) && (y0 === y1)) return;
        let e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx) { err += dx; y0 += sy; }
        requestAnimationFrame(draw);
    }
    draw();
}

#… continue
```

- Start the Drawing
  - Kicks off the drawing process.

- Visualization Example:
  - For a line from (1,1) to (4,3):
    - Draw (1,1)
    - Move to (2,2) [diagonal]
    - Move to (3,2) [horizontal]
    - Move to (4,3) [diagonal]

Colab: User-defined Functions

Module III: Computer Graphics

Step 4: Ending HTML with </html>

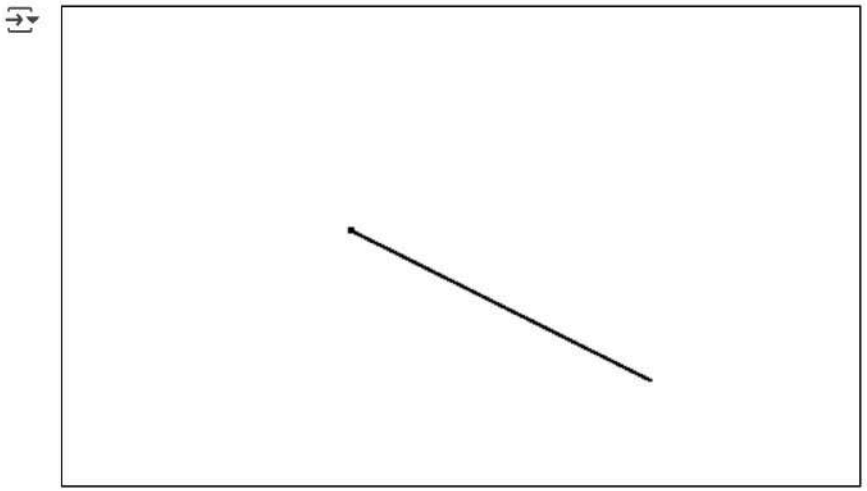
```
#… continue

</script>

</body>
</html>
"""
```

Step 5: Display an interactive screen

```
display(IPython.display.HTML(html_code))
```



Assignment: Line Drawing

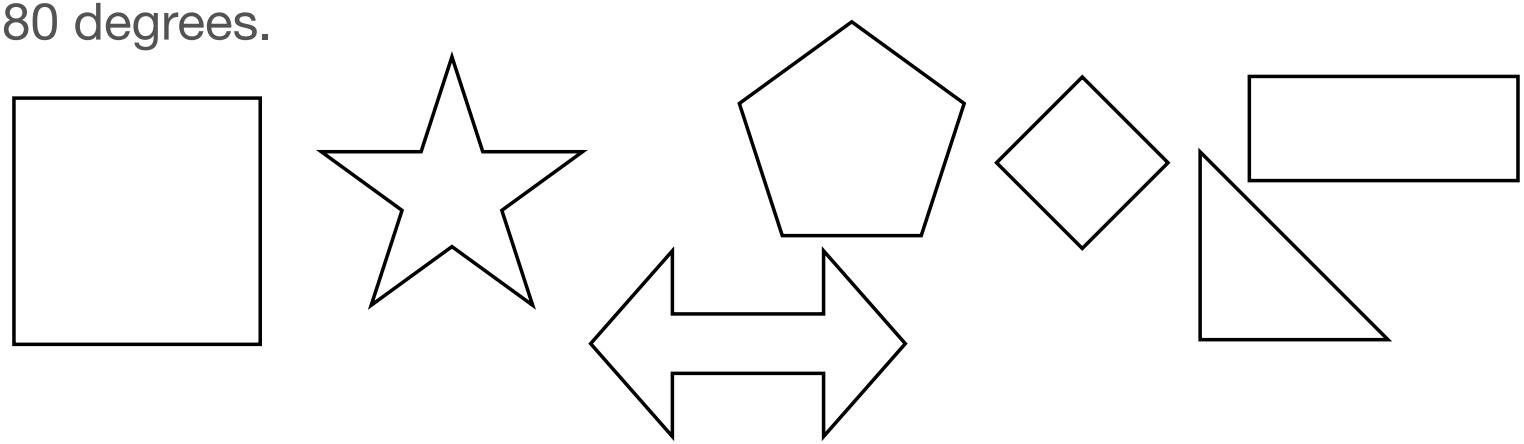
Module III: Drawing I

- Problems: (15 points)
  - Applying the DDA algorithm to draw a line by defining the start point and the endpoint. (5 points)
  - Modify the bresenhamLine( ) regarding the Bresenham algorithm in the lecture to draw a line by defining the start point and the endpoint. (5 points)
  - Applying the Midpoint Line Drawing algorithm to draw a line by defining the start point and the endpoint. (5 points)

Assignment: Line Drawing

Module III: Drawing I

- Problems: (25 points)
  - Applying an algorithm of line drawing to draw any closed shapes (25 points):
    - Closed shapes in geometry, like polygons, have angles formed at the points where their sides (line segments) meet, called vertices.
    - These angles are called interior angles, and their sum depends on the number of sides of the polygon. For example, a triangle has 3 angles that always add up to 180 degrees.



References

References

Reference Materials

- John F. Hughes, Andries van Dam, Morgan McGuire, David F. Sklar, James D. Foley, Steven K. Feiner, and Kurt Akeley, “Computer Graphics,” Third Edition, *Addison-Wesley*, 2014
- P. Joshi, D. M. Escriva and V. Godoy, “OpenCV by Example,” *Packt Publishing*, Birmingham - Mumbai, 297 pages, ISBN 978-1-78528-094-8, Jan 2016.
- Ioannis Deliyannis, “Interactive Multimedia,” *InTech*, Croatia, 2012
- Rajneesh Agrawal, “DCAP303:Multimedia Systems,” *Excel Books Private Limited*, LPU, Phagwara, India, 2013
- Sanjaya Mishra and Ramesh C. Sharma, “Interactive Multimedia in Education and Training,” *Idea Group Publishing*, Indira Gandhi National Open University, India, 2005
- Beat Games, http://beatsaber.com, Beat Saber, Accessed on Jul 29, 2025,
- Liv, “If You Want to ESCAPE with Me...Beat Saber ft SwanVR,” YouTube, Accessed on Jul 29, 2025





Computer Graphics: C++ Programming on Xcode

Computer Graphics Tools: Xcode

Module III: Computer Graphics

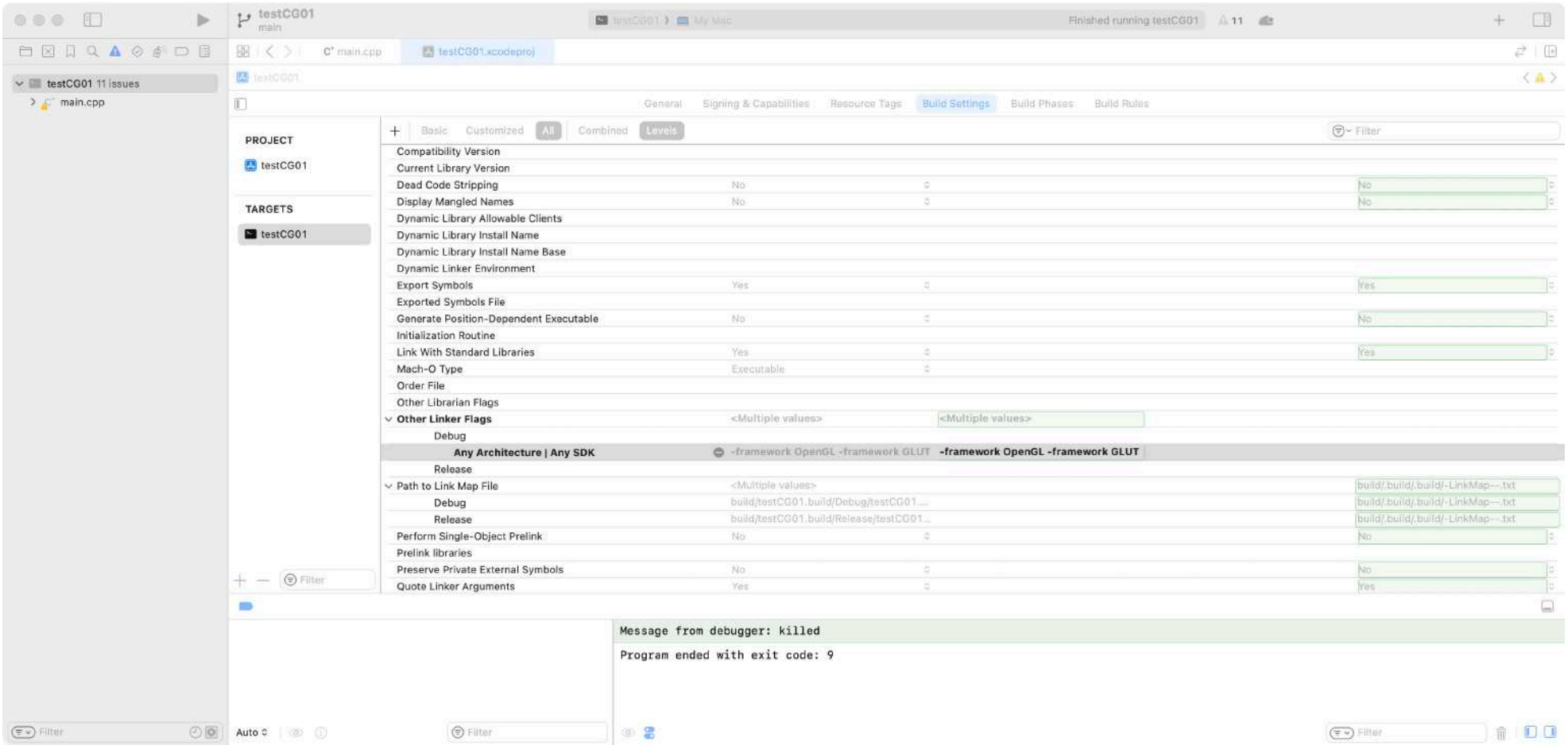
- Preparing Xcode

  Create a new Xcode Project:
  - macOS → Command Line Tool (C++).

  Configure OpenGL in Xcode:
  - Go to Build Settings → Other Linker Flags → Add:
    `-framework OpenGL -framework GLUT`

Xcode: Configuration

Module III: Computer Graphics



Xcode: Test using GL_POINTS

Module III: Computer Graphics

```
#include <OpenGL/gl.h>
#include <GLUT/glut.h>
#include <cmath>

// Function to plot a single pixel (dot) at (x, y)
void plotPoint(float x, float y) {
    glBegin(GL_POINTS);
        glVertex2f(x, y);
    glEnd();
}

// Bresenham's Line Algorithm (integer arithmetic)
void drawLine(int x0, int y0, int x1, int y1) {
    int dx = abs(x1 - x0);
    int dy = abs(y1 - y0);
    int sx = (x0 < x1) ? 1 : -1;
    int sy = (y0 < y1) ? 1 : -1;
    int err = dx - dy;

    while (true) {
        plotPoint(x0 / 400.0f - 1.0f, y0 / 300.0f - 1.0f); // Normalize to OpenGL coords [-1, 1]

        if (x0 == x1 && y0 == y1) break;

        int e2 = 2 * err;
        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }
        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}
```

Xcode: Test using GL_POINTS

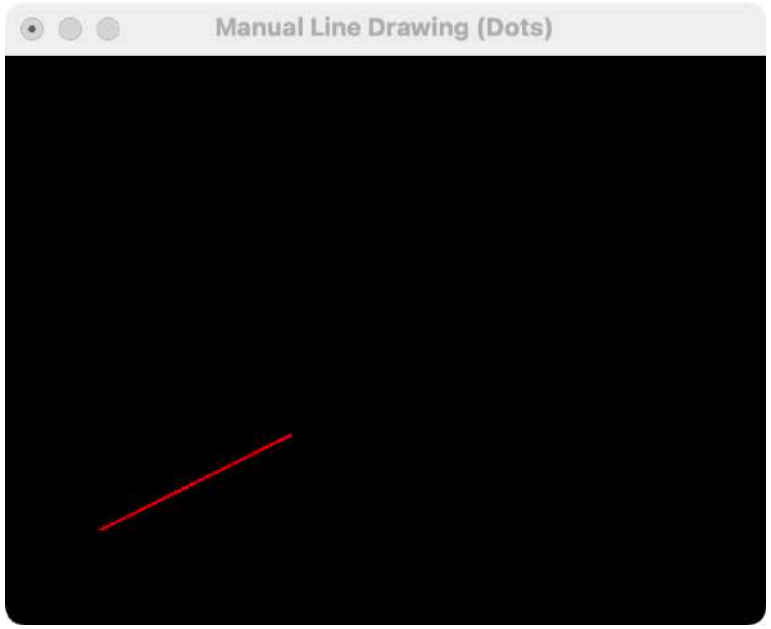
Module III: Computer Graphics

```
void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 0.0, 0.0); // Red

    // Draw a line from (100, 100) to (300, 200) in pixel coordinates
    drawLine(100, 100, 300, 200);

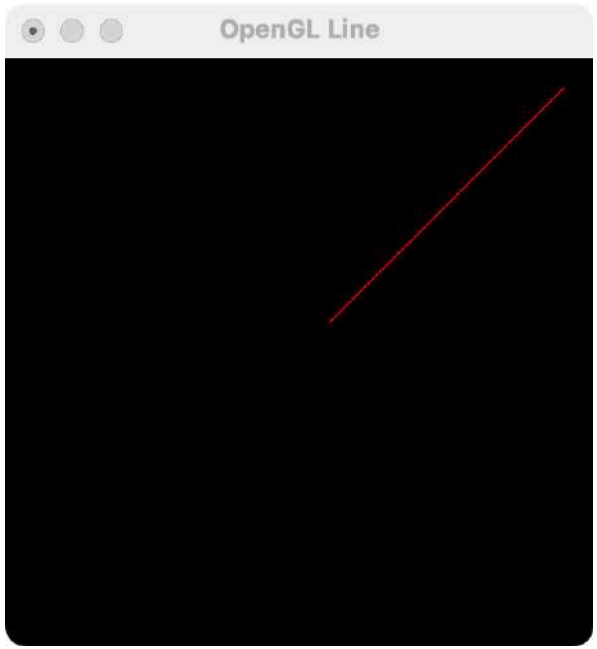
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(400, 300); // Window size (matches normalization in drawLine)
    glutCreateWindow("Manual Line Drawing (Dots)");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```



Xcode: Test using GL_LINES

Module III: Computer Graphics




```
#include <OpenGL/gl.h>
#include <GLUT/glut.h>

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw a red line from (0.1, 0.1) to (0.9, 0.9)
    glBegin(GL_LINES);
        glColor3f(1.0, 0.0, 0.0); // Red
        glVertex2f(0.1, 0.1);     // Start point
        glVertex2f(0.9, 0.9);     // End point
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutCreateWindow("OpenGL Line");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Computer Graphics C++ Programming on Qt Creator

Qt Creator: QPainter: painter.drawLine( )

Module III: Computer Graphics

Step 1: Add OpenGL to `.pro` File

```
QT += core gui
QT += opengl

greaterThan(QT_MAJOR_VERSION, 4): QT += widgets

CONFIG += c++17

# You can make your code fail to compile if it uses deprecated APIs.
# In order to do so, uncomment the following line.
# DEFINES += QT_DISABLE_DEPRECATED_BEFORE=0x060000
# disables all the APIs deprecated before Qt 6.0.0

SOURCES += \
    main.cpp \
    mainwindow.cpp

HEADERS += \
    mainwindow.h

FORMS += \
    mainwindow.ui

# Default rules for deployment.
qnx: target.path = /tmp/$${TARGET}/bin
else: unix:!android: target.path = /opt/$${TARGET}/bin
!isEmpty(target.path): INSTALLS += target
```

Qt Creator: QPainter: painter.drawLine( )

Module III: Computer Graphics

Step 2: Modify `mainwindow.h`

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QPainter>

class MainWindow : public QMainWindow {
    Q_OBJECT
public:
    MainWindow(QWidget *parent = nullptr);
    void paintEvent(QPaintEvent *event) override;  // Override paint event
};

#endif // MAINWINDOW_H
```

Qt Creator: QPainter: painter.drawLine( )

Module III: Computer Graphics

Step 3: Implement Line Drawing in `mainwindow.cpp`

```cpp
#include <QMainWindow>
#include <QPainter>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
    setFixedSize(400, 300);  // Set window size
}

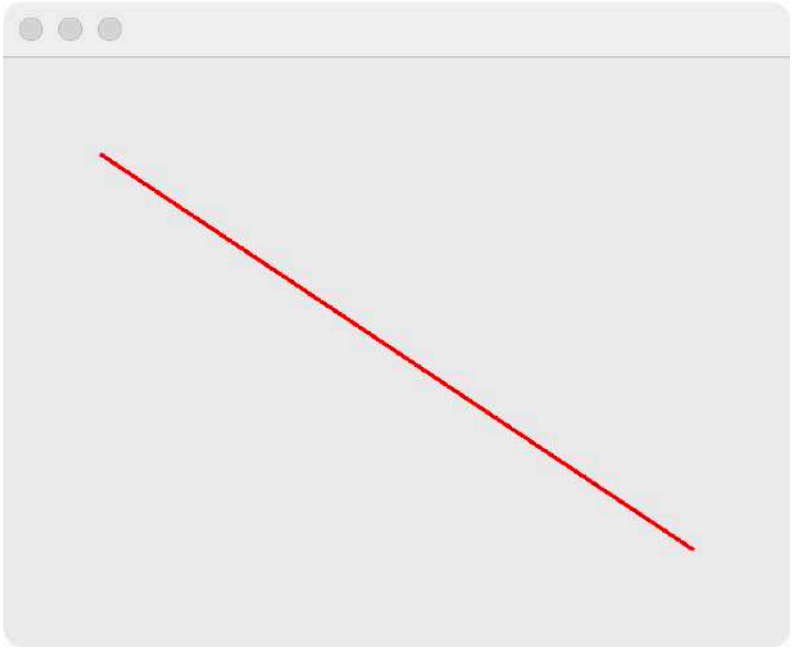
void MainWindow::paintEvent(QPaintEvent *event) {
    Q_UNUSED(event);
    QPainter painter(this);
    painter.setPen(QPen(Qt::red, 2));  // Red line, 2px thick

    // Draw line from (50, 50) to (350, 250)
    painter.drawLine(50, 50, 350, 250);
}
```

Qt Creator: QPainter: painter.drawLine( )

Module III: Computer Graphics

Step 4: Look around in `main.cpp`



```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return a.exec();
}
```

Qt Creator: User-defined Functions

Module III: Computer Graphics

Step 1: Add OpenGL to `.pro` File

```
QT       += core gui

greaterThan(QT_MAJOR_VERSION, 4): QT += widgets

CONFIG += c++17

# You can make your code fail to compile if it uses deprecated APIs.
# In order to do so, uncomment the following line.
#DEFINES += QT_DISABLE_DEPRECATED_BEFORE=0x060000    # disables all the APIs deprecated before Qt 6.0.0

SOURCES += \
    main.cpp \
    mainwindow.cpp

HEADERS += \
    mainwindow.h

FORMS += \
    mainwindow.ui

# Default rules for deployment.
qnx: target.path = /tmp/$${TARGET}/bin
else: unix:!android: target.path = /opt/$${TARGET}/bin
!isEmpty(target.path): INSTALLS += target
```

Qt Creator: User-defined Functions

Module III: Computer Graphics

Step 2: Modify `mainwindow.h`

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QImage>

class MainWindow : public QMainWindow {
    Q_OBJECT
public:
    MainWindow(QWidget *parent = nullptr);
    void paintEvent(QPaintEvent *event) override;

private:
    QImage m_image;  // Acts as a pixel buffer
    void drawLine(int x0, int y0, int x1, int y1);
    void plotPoint(int x, int y);
};

#endif // MAINWINDOW_H
```

Qt Creator: User-defined Functions

Module III: Computer Graphics

Step 3: Implement Line Drawing in `mainwindow.cpp`

```
#include "mainwindow.h"
#include <QPainter>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
    setFixedSize(400, 300);  // Window size

    // Initialize QImage (pixel buffer)
    m_image = QImage(400, 300, QImage::Format_ARGB32);
    m_image.fill(Qt::black);  // Black background

    // Draw a line using manual pixel plotting
    drawLine(50, 50, 350, 250);
}

void MainWindow::paintEvent(QPaintEvent *event) {
    Q_UNUSED(event);
    QPainter painter(this);
    painter.drawImage(0, 0, m_image);  // Draw the pixel buffer
}

...
```

Qt Creator: User-defined Functions

Module III: Computer Graphics

Step 3: Implement Line Drawing in `mainwindow.cpp`

```
...

// Bresenham's line algorithm (pixel-based)
void MainWindow::drawLine(int x0, int y0, int x1, int y1) {
    int dx = abs(x1 - x0);
    int dy = abs(y1 - y0);
    int sx = (x0 < x1) ? 1 : -1;
    int sy = (y0 < y1) ? 1 : -1;
    int err = dx - dy;

    while (true) {
        plotPoint(x0, y0);  // Plot the current pixel

        if (x0 == x1 && y0 == y1) break;

        int e2 = 2 * err;
        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }
        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}
...
```

Qt Creator: User-defined Functions

Module III: Computer Graphics

Step 3: Implement Line Drawing in `mainwindow.cpp`

```
...

// Plot a single pixel (using QImage)
void MainWindow::plotPoint(int x, int y) {
    if (x >= 0 && x < m_image.width() && y >= 0 && y < m_image.height()) {
        m_image.setPixel(x, y, qRgb(255, 0, 0));  // Red pixel (RGB)
    }
}
```

Qt Creator: User-defined Functions

Module III: Computer Graphics

Step 4: Look around in `main.cpp`

```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return a.exec();
}
```

